

Block Drill Exam — Set 2

Twelve more short blocks; same method: card line by line, notes, findings, verdict; answers follow each block

October 2026

How to run it. Same as Set 1: requirement, then the card on every numbered line, write the notes, list findings from the notes, give confidence and verdict, then uncover. 8 minutes for 1–6, 12 for 7–12. Assume `async` for any I/O-named function.

Block 1 — Unique emails

Requirement:

return the list of distinct emails, lowercased, in first-seen order; input must not change.

```
1 function uniqueEmails(users: User[]): string[] {
2   const seen = [];
3   for (const u of users) {
4     u.email = u.email.toLowerCase();
5     if (!seen.includes(u.email)) seen.push(u.email);
6   }
7   return seen;
8 }
```

Notes

Line	Note
2	<code>seen</code> holds a new array; typed <code>any[]</code> (no annotation on an empty literal)
3	<code>u</code> shared with <code>users[]</code>
4	MUTATES each user's email; <code>u.email.toLowerCase</code> crashes if <code>email</code> is <code>undefined</code>
5	<code>includes</code> in a loop: $O(n^2)$; correct logic, first-seen order preserved
7	returns <code>seen</code>

Findings

- Line 4: mutates the caller's user objects; requirement says input must not change; use a local `const e = u.email.toLowerCase()`.
- Line 4: crashes if `email` is optional/undefined; guard or filter first.
- Line 5: `includes` inside the loop is $O(n^2)$; a `Set` keeps order via insertion and is $O(n)$ (`[...new Set(...)]`); minor unless large.
- Line 2: `const seen = []` infers `any[]`; annotate `string[]`.

Confidence 2 • Request changes.

Output is correct; the input-mutation clause fails.

Block 2 — Delete endpoint

Requirement: `DELETE /items/:id` removes the item if it belongs to the caller; 403 otherwise; 404 if absent; 204 on success.

```
1 app.delete("/items/:id", async (req, res) => {
2   const item = await db.item.findUnique({ where: { id: req.params.id } });
3   if (!item) return res.status(404).end();
4   if (item.ownerId !== req.body.userId) return res.status(403).end();
5   await db.item.delete({ where: { id: item.id } });
6   res.status(204);
7 });
```

Notes

Line	Note
2	awaited; <code>item</code> holds a record or <code>null</code>
3	<code>!item</code> correct for <code>null</code> ; returns
4	ownership compared against <code>req.body.userId</code> : the CALLER supplies their own identity; also a DELETE often has no body
5	awaited delete
6	sets the status but never sends: <code>res.status(204)</code> without <code>.end()</code> / <code>.send()</code> ; the request hangs

Findings

- 1. Line 4: identity from `req.body` is attacker-controlled; must come from the verified token (`req.user.id` or claims).
- 2. Line 6: response never sent; add `.end()`.
- 3. Line 1: async handler in Express 4; a thrown DB error hangs the request (design note; wrap or `next(e)`).

Confidence 1 • Reject.

Authorization bypass.

Block 3 — Parse duration

Requirement: parse "90s", "5m", "2h" to milliseconds; throw on anything else.

```
1 function parseDuration(s: string): number {
2   const n = parseInt(s);
3   const unit = s[s.length - 1];
4   const mult = { s: 1000, m: 60_000, h: 3_600_000 }[unit];
5   return n * mult;
6 }
```

Notes

Line	Note
2	<code>parseInt("90s")</code> is 90 (fine); <code>parseInt("abc")</code> is NaN; <code>parseInt("")</code> is NaN; no radix (nit)
3	last char; <code>" "</code> gives <code>undefined</code>
4	object lookup by unit; unknown unit gives <code>undefined</code> ; <code>"constructor"</code> is not reachable since <code>unit</code> is one char
5	<code>NaN * anything</code> is NaN; <code>n * undefined</code> is NaN; never throws

Findings

- Lines 4–5: unknown unit or non-numeric prefix yields NaN instead of throwing; check `mult === undefined` and `Number.isNaN(n)` and throw.
- Line 2: `"5.5m"` parses to 5; `"-5m"` to -5; negative and fractional are not rejected.
- Line 2: `parseInt` without radix (nit).

Confidence 2 • Request changes.

Happy path works; the throw clause is entirely absent.

Block 4 — Batch save

Requirement:

save all records; if any fails, throw after all have been attempted, listing the failed ids.

```
1  async function saveAll(records: Rec[]): Promise<void> {
2    const failed: string[] = [];
3    await Promise.all(records.map(async (r) => {
4      try { await save(r); } catch { failed.push(r.id); }
5    }));
6    if (failed) throw new Error(`failed: ${failed.join(",")}`);
7  }
```

Notes

Line	Note
3	all saves in parallel, each wrapped; <code>Promise.all</code> cannot reject because every callback catches
4	<code>await</code> inside the <code>try</code> : correct; push to shared array from concurrent callbacks is safe (single thread)
6	<code>if (failed)</code> : an array is ALWAYS truthy; throws every time, even with zero failures

Findings

1. Line 6: `failed` is an array; `[]` is truthy; should be `failed.length > 0`.

Positives: parallel with per-item catch so all are attempted; `await` inside `try`.

Confidence 1 · Request changes.

One-line bug, but it fires on every call.

Block 5 — Group by key

Requirement: group items by `item.category`; return `Record<string, Item[]>`; categories come from user input.

```
1 function groupBy(items: Item[]): Record<string, Item[]> {
2   const out: Record<string, Item[]> = {};
3   for (const it of items) {
4     out[it.category] ??= [];
5     out[it.category].push(it);
6   }
7   return out;
8 }
```

Notes

Line	Note
2	plain object with user-controlled keys
4	<code>??=</code> creates the array if nullish; for <code>"constructor"</code> , <code>out["constructor"]</code> is an inherited FUNCTION (not nullish), so no array is created
5	<code>out["constructor"].push</code> → <code>TypeError</code> ; <code>"__proto__"</code> as a category assigns onto the prototype
7	returns the object; elements shared with <code>items</code> (fine)

Findings

- Lines 2–5: plain object with user-controlled keys; `"constructor"` crashes, `"__proto__"` pollutes. Use `new Map<string, Item[]>()` or `Object.create(null)`.

Positives: `??=` is the right idiom for a trusted-key object.

Confidence 3 · Request changes.

Works for ordinary categories; fails on adversarial ones, which the requirement explicitly allows.

Block 6 — Debounced search

Requirement: call `search(q)` at most once per 300 ms of typing; always with the latest query.

```
1 let timer: number | undefined;
2 function onInput(q: string) {
3   if (timer) return;
4   timer = setTimeout(() => {
5     search(q);
6     timer = undefined;
7   }, 300);
8 }
```

Notes

Line	Note
3	if a timer is pending, IGNORE this call: the query <code>q</code> of later keystrokes is dropped
5	<code>search(q)</code> uses the <code>q</code> captured at the FIRST call, not the latest
6	resets after firing
3	also <code>if (timer)</code> is falsy for id 0 (browsers can return 0? no, ids start at 1; Node returns an object; fine)

Findings

- Lines 3–5: this is a leading-edge throttle that drops later input; the requirement is a trailing debounce with the latest query. Clear and re-arm on every call, and read the latest `q` from a variable updated on each call.
- Line 5: `search(q)` is async (I/O); not awaited and no `.catch`; rejections unhandled (minor here).

Confidence 1 • Reject.

Never delivers the latest query.

Block 7 — Inventory reserve

Requirement: `reserve(sku, qty)` decrements stock if enough is available, returns `true`; else returns `false` without changing stock; must be safe under concurrent calls.

```
1  const stock = new Map<string, number>();
2  async function reserve(sku: string, qty: number): Promise<boolean> {
3    const have = stock.get(sku) ?? 0;
4    if (have < qty) return false;
5    await audit.log("reserve", sku, qty);
6    stock.set(sku, have - qty);
7    return true;
8  }
```

Notes

Line	Note
3	read of shared state
4	boundary: <code>have < qty</code> rejects when short; <code>have === qty</code> allowed; correct; <code>qty <= 0</code> or <code>NaN</code> not validated (<code>NaN</code> passes: <code>have < NaN</code> is false)
5	AWAIT between read and write: another <code>reserve</code> for the same sku can read the same <code>have</code>
6	writes <code>have - qty</code> using the stale <code>have</code> ; two concurrent reserves of 5 from stock 5 both succeed and stock becomes 0 (or negative on a third)

Findings

- Lines 3–6: read-await-write on shared stock; concurrent calls over-reserve. Move the write before the `await` (decrement synchronously, then log), or serialize per sku.
- Line 4: `qty` not validated; `NaN`, `0`, negative pass; negative `qty` INCREASES stock at line 6.

Positives: `?? 0` for unknown sku; correct comparison direction.

Confidence 2 · Request changes.

The explicitly required concurrency clause fails.

Block 8 — Config merge with env

Requirement: build config from defaults, then a JSON file, then env vars (`PORT`, `DEBUG`), later sources winning; `PORT` is an integer, `DEBUG` a boolean.

```
1  const DEFAULTS = { port: 3000, debug: false, tags: [] as string[] };
2  function buildConfig(fileJson: string) {
3    const file = JSON.parse(fileJson);
4    const cfg = Object.assign(DEFAULTS, file);
5    if (process.env.PORT) cfg.port = process.env.PORT;
6    if (process.env.DEBUG) cfg.debug = process.env.DEBUG === "true";
7    return cfg;
8  }
```

Notes

Line	Note
3	unguarded parse, <code>any</code>
4	<code>Object.assign(DEFAULTS, ...)</code> MUTATES the module-level <code>DEFAULTS</code> ; <code>cfg === DEFAULTS</code> ; every later call starts from the previous file's values; <code>DEFAULTS.tags</code> array is shared too
5	<code>process.env.PORT</code> is a string; assigned to <code>cfg.port</code> (typed number) via <code>any</code> leakage from line 3; <code>"3000"</code> stored, not 3000
6	<code>DEBUG="false"</code> → <code>false</code> (correct); <code>DEBUG="1"</code> → <code>false</code> (arguably wrong); <code>DEBUG=""</code> is falsy so ignored (fine)
7	returns the mutated defaults

Findings

- 1. Line 4: `Object.assign` with `DEFAULTS` as target mutates shared state; use `{ ...DEFAULTS, ...file }` (and copy `tags`).
- 2. Line 5: `PORT` stored as a string; `Number(...)` with an integer/range check.
- 3. Line 3: unguarded `JSON.parse`; unvalidated shape.
- 4. Line 6: only `"true"` counts as true; document or accept `"1"` (nit).

Confidence 1 • Reject.

Shared-state corruption plus a type violation on the first clause.

Block 9 — Stream lines

Requirement:

read a file line by line, count non-empty lines, resolve with the count; the file may be large.

```
1 function countLines(path: string): Promise<number> {
2   let n = 0;
3   const rl = readline.createInterface({ input: fs.createReadStream(path) });
4   rl.on("line", (line) => { if (line.trim()) n++; });
5   rl.on("close", () => n);
6   return new Promise((resolve) => resolve(n));
7 }
```

Notes

Line	Note
3	stream created; reading starts asynchronously
4	listener increments <code>n</code> per line, later
5	<code>close</code> listener returns <code>n</code> to nobody; a listener's return value is discarded
6	resolves IMMEDIATELY with the current <code>n</code> , which is 0, before any line has been read

Findings

- Lines 5–6: the Promise resolves synchronously with 0; `resolve` must be called inside the `close` handler: `return new Promise((resolve, reject) => { rl.on("close", () => resolve(n)); rl.on("error", reject); })`.
- No `error` listener on the stream or interface; a missing file crashes the process with an unhandled `error` event.

Confidence 1 · Reject.

Always returns 0.

Block 10 — Pagination cursor

Requirement: return { items, nextCursor }; nextCursor is the last item's id, or null when there are no more; page size limit ≤ 100.

```
1  async function page(cursor: string | null, limit: number) {
2    const rows = await db.query(
3      `SELECT * FROM t WHERE id > '${cursor ?? ""}' ORDER BY id LIMIT ${limit}`
4    );
5    const nextCursor = rows.length === limit ? rows[rows.length - 1].id : null;
6    return { items: rows, nextCursor };
7  }
```

Notes

Line	Note
3	cursor interpolated into SQL: injection; limit interpolated: injection if not validated; limit never checked against 100
5	rows.length === limit means "maybe more"; if exactly limit remain, one extra empty page is returned (acceptable; say so); rows[rows.length - 1] safe because length === limit > 0 unless limit is 0
5	limit = 0: rows.length === 0 === limit → rows[-1].id → crash

Findings

- 1. Line 3: SQL injection via cursor and limit; parameterize (\$1, \$2).
- 2. Line 1/3: limit not validated against 1–100; limit = 0 crashes line 5; limit = 1e9 dumps the table.
- 3. Line 5: exact-boundary extra page is a known trade-off; note, not a finding.

Confidence 1 · Reject.

Injection.

Block 11 — Observable value

Requirement: `Value<T>` holds a value; `subscribe(fn)` calls `fn` on every change and returns an unsubscribe; `set(v)` notifies only if the value changed.

```
1 class Value<T> {
2   private subs: Array<(v: T) => void> = [];
3   constructor(private v: T) {}
4   get(): T { return this.v; }
5   set(v: T) {
6     if (v == this.v) return;
7     this.v = v;
8     for (const s of this.subs) s(v);
9   }
10  subscribe(fn: (v: T) => void) {
11    this.subs.push(fn);
12    return () => this.subs.splice(this.subs.indexOf(fn), 1);
13  }
14 }
```

Notes

Line	Note
4	returns the stored value itself; if <code>T</code> is an object, the caller holds an alias and can mutate without <code>set</code> (design note)
6	<code>==</code> loose; for primitives mostly fine; for objects it is identity, so <code>set({...})</code> always notifies; <code>set(0)</code> after <code>""</code> does NOT notify (<code>0 == ""</code>)
8	iterates the live <code>subs</code> array; a subscriber that unsubscribes during notification splices the array mid-loop and the next subscriber is skipped
12	<code>indexOf</code> returns -1 if already removed; <code>splice(-1, 1)</code> removes the LAST subscriber (someone else's)

Findings

1. Line 12: double-unsubscribe removes a different subscriber; guard `if (i !== -1)`.
2. Line 8: iterate a copy `[...this.subs]` so unsubscribe-during-emit does not skip.
3. Line 6: `==` coercion; use `===` (or `Object.is` for `NaN`).
4. Line 4: returns an alias for object `T` (design note).

Confidence 2 · Request changes.

Block 12 — The clean one

Requirement: `once(fn)` returns a function that calls `fn` the first time and returns its result; later calls return the cached result without calling `fn`; must work for async `fn` and must not cache a rejection.

```
1 function once<A extends unknown[], R>(fn: (...a: A) => Promise<R>) {
2   let pending: Promise<R> | null = null;
3   return (...args: A): Promise<R> => {
4     if (pending === null) {
5       pending = fn(...args).catch((e) => { pending = null; throw e; });
6     }
7     return pending;
8   };
9 }
```

Notes

Line	Note
2	one closure variable, <code>null</code> until first call
4	<code>=== null</code> check, not truthy
5	assigns the Promise synchronously, so concurrent first calls share it; the <code>.catch</code> resets to <code>null</code> on rejection so a retry is possible, then rethrows so the caller still sees the error
7	returns the same Promise to every caller

Findings

- Nit: later calls with different `args` are ignored (by design of `once`; say so).
- Nit: a synchronous throw inside `fn` (before it returns a Promise) is not caught by `.catch`; wrap in `Promise.resolve().then(() => fn(...args))` if that matters.

Positives:

shared in-flight Promise; rejection not cached; rethrow preserves the error; generic types carry through.

Confidence 5 · Approve.

Scoring yourself

Block	Must-find
1	mutation of input (4)
2	identity from body (4), response never sent (6)
3	never throws (4–5)
4	<code>if (failed)</code> on an array (6)
5	plain-object user keys (2–5)
6	throttle not debounce; stale <code>q</code> (3–5)
7	read-await-write (3–6)
8	<code>Object.assign</code> on defaults (4), <code>PORT</code> string (5)
9	resolves immediately with 0 (6)
10	SQL injection (3)
11	<code>splice(-1)</code> on double unsubscribe (12), live-array iteration (8)
12	nothing; approve